

Supervised Learning Setup — Foundations for Neural Networks

1. What Does It Mean to Learn From Data?

In supervised learning, we want to learn a **mapping** from inputs to outputs using labeled examples.

Each dataset contains:

- **Features (X):** the input variables
- **Targets (y):** the desired output

Mathematically: $\hat{y} = f_{\theta}(x)$

Where:

- f_{θ} is the model (a neural network)
- θ represents the learnable parameters (weights and biases)

Goal:

Learn parameters that produce accurate predictions on unseen data.

2. Types of Supervised Learning Tasks

Regression

- Output is a continuous value.
- Examples: price prediction, temperature forecasting.

Classification

- Output is a category or label.
- Binary classification: two classes.
- Multi-class classification: more than two classes.

Important idea:

- The **task type** influences the choice of activation functions and loss functions later.
-

3. Dataset Structure and Shapes

We organize data as:

- $X \in \mathbb{R}^{(n_{samples}, n_{features})}$
- $y \in \mathbb{R}^{(n_{samples})}$

Interpretation:

- Each **row** = one sample.
- Each **column** = one feature.

Understanding shapes is essential because neural networks rely heavily on matrix operations.

4. Train / Validation / Test Split

To evaluate learning properly, we divide the data into:

- **Training set:** used to learn parameters.
- **Validation set:** used to tune decisions and monitor performance.
- **Test set:** used only for final evaluation.

Key intuition:

A good model should perform well on data it has never seen before.

5. The Supervised Learning Loop

All supervised models follow the same cycle: Input → Model → Prediction → Loss → Parameter Update → Repeat

Steps:

1. Feed input X into the model.
2. Produce prediction $\hat{y}$.
3. Measure error using a loss function.
4. Update parameters to reduce the error.

Neural networks will follow this exact workflow — just with more complex models.

6. What Is a Loss Function? (Conceptual View)

A **loss function** measures how wrong a prediction is.

Examples:

- Regression: distance between predicted and true values.
- Classification: how incorrect predicted probabilities are.

Lower loss $\Rightarrow$ better predictions.

We will formally define specific loss functions later in the course.

7. Generalization vs Overfitting (Intuition)

Two possible outcomes when training a model:

- **Generalization:** model learns patterns that apply to new data.
- **Overfitting:** model memorizes training data but fails on unseen data.

Goal of supervised learning:

Learn patterns, not memorize examples.

8. Why Do We Need Neural Networks?

Linear models can only learn simple relationships.

Many real-world problems require:

- Non-linear decision boundaries
- Hierarchical feature transformations

Neural networks extend supervised learning by introducing:

- Multiple layers
- Non-linear activation functions

This allows models to learn complex mappings from input to output.

Python Revision

NumPy

In Python we have lists that serve the purpose of arrays, but they are slow to process. NumPy aims to provide an array object that is up to 50x faster than traditional Python lists. Why we prefer numpy arrays:

- Fast operations
- Vectorized math
- Supports matrix multiplication

```
import numpy as np # commonly used alias
import matplotlib.pyplot as plt

np.random.seed(42) # for reproducibility (so results are the same every run)
```

```
lst = [1, 2, 3]
arr = np.array([1, 2, 3])

print(type(lst), type(arr))

<class 'list'> <class 'numpy.ndarray'>
```

Shapes

```
x = np.array([1, 2, 3])
print(x.shape)

(3,)
```

column vector

```
x_col = x.reshape(-1, 1)
print(x_col.shape)

(3, 1)
```

row vector

```
x_row = x.reshape(1, -1)
print(x_row.shape)

(1, 3)
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr2 = arr.reshape(4, 3)
newarr3 = arr.reshape(2, 3, 2)
print(newarr2)
print("-----")
print(newarr3)
```

```
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
-----
[[[ 1  2]
 [ 3  4]
 [ 5  6]]

 [[ 7  8]
 [ 9 10]
 [11 12]]]
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
newarr = arr.reshape(3, 3)
```


ValueError

Traceback (most recent call

```

last)
<ipython-input-16-cf832910fe8f> in <module>
      1 arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
----> 2 newarr = arr.reshape(3, 3)

ValueError: cannot reshape array of size 12 into shape (3,3)

arr = np.array([[1, 2, 3], [4, 5, 6]])
newarr = arr.reshape(-1) # flatten
print(newarr)

[1 2 3 4 5 6]

```

Indexing and Slicing

```

X = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

print(X[0])
print(X[:, 0])
print(X[0:2, 1:3])

[1 2 3]
[1 4 7]
[[2 3]
 [5 6]]

```

Elementwise Operations

```

a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

print(a + b)
print(a * b)

[11 22 33]
[10 40 90]

```

Broadcasting

```

X = np.array([[1, 2],
              [3, 4],
              [5, 6]])

b = np.array([10, 100])
print(X + b)

```

```
[[ 11 102]
 [ 13 104]
 [ 15 106]]
```

Dot Product and Matrix Multiplication

```
x = np.array([1, 2, 3])
w = np.array([0.1, 0.2, 0.3])

print(np.dot(x, w))

1.4

X = np.array([[1, 2],
              [3, 4],
              [5, 6]])

W = np.array([[0.1],
              [0.2]])

Z = X @ W
print(Z.shape)
print(Z)

(3, 1)
[[0.5]
 [1.1]
 [1.7]]

b = np.array([0.5])
print(Z + b)

[[1. ]
 [1.6]
 [2.2]]

A = np.array([[3, 2, 4], [5, 0, 1], [3, 5, 2]])
print(A)

[[3 2 4]
 [5 0 1]
 [3 5 2]]

A.transpose()

array([[3, 5, 3],
       [2, 0, 5],
       [4, 1, 2]])

A.dot(A.transpose()) # AA'
```

```

array([[29, 19, 27],
       [19, 26, 17],
       [27, 17, 38]])

A.transpose().dot(A) # A'A
array([[43, 21, 23],
       [21, 29, 18],
       [23, 18, 21]])

A.mean() # same as np.mean(A)
2.7777777777777777

A.mean(axis=0) # row wise
array([3.66666667, 2.33333333, 2.33333333])

A.mean(axis=1) # column wise
array([3.        , 2.        , 3.33333333])

```

OOP & Clean Code

Functions

```

def my_func():
    print("This is my function")

my_func()

This is my function

def my_func(name):
    print(f"This is {name}'s function")

my_func() # this function expects an argument
-----
-----
TypeError                                Traceback (most recent call
last)
Cell In[25], line 1
----> 1 my_func()

TypeError: my_func() missing 1 required positional argument: 'name'

def my_func(name="Hagar"): # parameter: name, argument: "Hagar"
    print(f"This is {name}'s function")

my_func()

```

```

This is Hagar's function
my_func("Amr")
This is Amr's function
def my_func(name, age):
    print(f"This is {name} and I'm {age} years old")
my_func(23, "Hagar")
This is 23 and I'm Hagar years old
my_func(age = 23, name = "Hagar") # kwargs
This is Hagar and I'm 23 years old
def my_func(x,y):
    return x*y
my_func(4,5)
20

```

Doc String

```

intro = """
hi my name
is hagar
how are you?
"""

print(intro)

hi my name
is hagar
how are you?

```

A **docstring** is a short text written at the top of a function for reusability purposes that explains:

- What the function does
- What inputs it expects
- What it returns

```

def add_2_numbers(a, b = 1):
    """
    This function adds 2 input numbers `a` and `b` and assigns the
    result to variable `c`.

    Parameters:

```



```

    a : first input number with no default value
    b : second input number with a default value of 1

Returns:
    c : the sum of `a` and `b`

"""
c = a + b
return c
def linear_forward(X, W, b):
    """
    Computes the linear forward pass.

    Parameters:
        X (np.ndarray): Input data of shape (n_samples, n_features)
        W (np.ndarray): Weight matrix of shape (n_features, n_units)
        b (np.ndarray): Bias vector of shape (n_units,)

    Returns:
        np.ndarray: Output Z of shape (n_samples, n_units)
    """
    return X @ W + b

np.random.seed(0)
X = np.random.randn(4, 3)      # 4 samples, 3 features
W = np.random.randn(3, 2)      # 2 neurons
b = np.zeros(2)

Z = linear_forward(X, W, b)
print(Z.shape)

(4, 2)

```

Classes and Objects

```

class MyClass:
    x = 5

p1 = MyClass()
print(p1.x)

5

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

p1 = Person("John", 36)

```

```
print(p1.name)
print(p1.age)

John
36

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def myfunc(self):
        print("Hello my name is " + self.name)

p1 = Person("John", 36)
p1.myfunc()

Hello my name is John

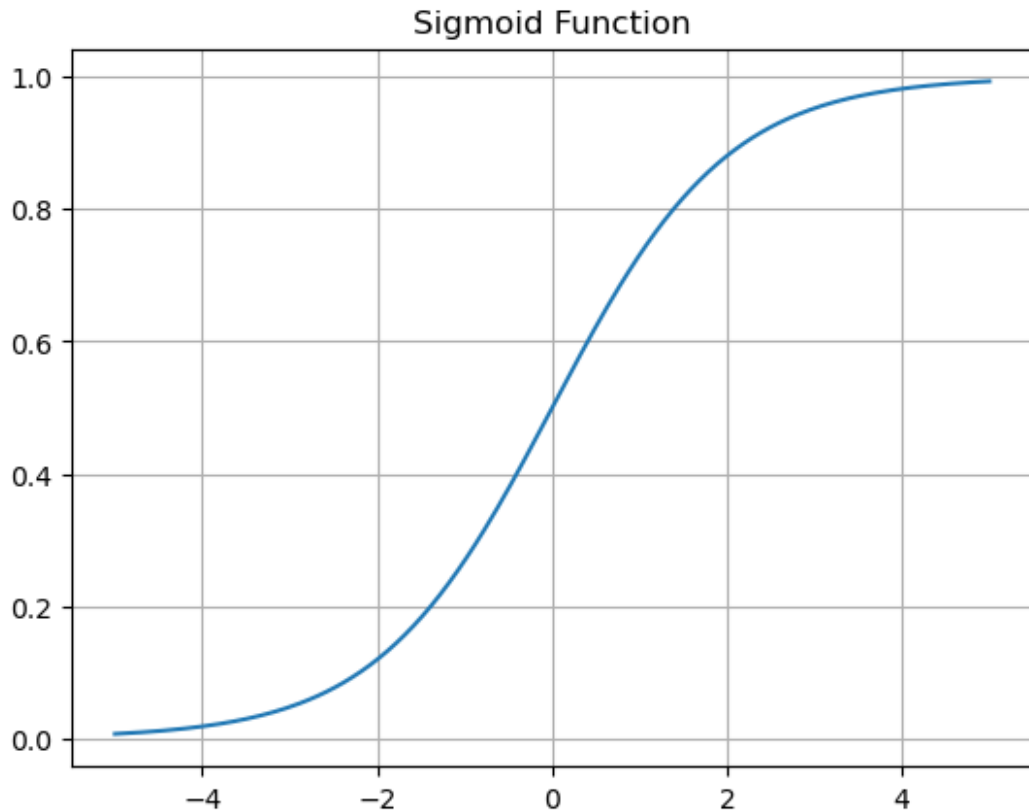
p1.age = 40
print(p1.age)

40
```

Plotting

```
x = np.linspace(-5, 5, 200)
sigmoid = 1 / (1 + np.exp(-x))

plt.plot(x, sigmoid)
plt.title("Sigmoid Function")
plt.grid(True)
plt.show()
```



Task

Your goal is to create a simple class that performs **matrix multiplication**, similar to what happens inside a neural network layer.

Create a class called `MatrixMultiplier`.

The class must contain:

1. An `__init__` function

The constructor should:

- Receive a matrix `W`
- Store it inside the object as an attribute

2. A method that performs matrix multiplication called `forward(self, X)`

This function should:

- Take an input matrix `X`
- Return the result of: `X · W`

```
# Test:
X = np.array([
    [1, 2],
    [3, 4],
    [5, 6]
])

W = np.array([
    [0.1],
    [0.2]
])

layer = MatrixMultiplier(W)

output = layer.forward(X)

print("Output:")
print(output)
print("Output shape:", output.shape)
```